International Journal of Machine Learning and Cybernetics
https://doi.org/10.1007/s13042-025-02544-w

ORIGINAL ARTICLE

Enhancing network security in industrial IoT environments: a DeepCLG hybrid learning model for cyberattack detection

Qawsar Gulzar[1] · **Khuram Mustafa**[1]

Received: 6 June 2024 / Accepted: 7 January 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract
The Industrial Internet of Things (IIoT) combines sensors, machinery, industrial software, databases, services, and personnel within the computing environment. IIoT enhances cities, agriculture, e-healthcare, and other areas of life. While IIoT and IoT share many characteristics, they deploy different cybersecurity methods for their networks. Unlike consumer IoT solutions, Industrial Internet of Things (IIoT) solutions typically integrate into operational systems instead of operating independently. Previous investigations suffered a major weakness when they failed to use sampling techniques to balance the datasets from real IIoT operations, which comprise multiple attack types. This can result in models performing poorly on minority classes but well on majority classes, limiting the intrusion detection system's overall effectiveness. Therefore, security solutions for IIoT require extra strategic planning and continuous monitoring to protect system security and privacy. The objective is to identify traffic data irregularities from IIoT networks. This research introduces a deepCLG hybrid learning model designed to improve network intrusion detection systems (NIDSs). The datasets first go through preprocessing and normalization. Following this, we then formulated a hybrid learning model named DeepCLG. It integrates the convolutional neural network (CNN), the long short-term memory (LSTM), and the gated recurrent unit (GRU) with the capsule network (CN). The model's efficacy is assessed using two widely accessible datasets: the CICIoT 2023 and UNSW_NB15 datasets. The proposed model outperformed existing techniques in detecting attacks and achieved an accuracy of 99.82%, precision of 97.83%, detection rate of 95.91%, Matthew's correlation coefficient (MCC) of 96.77%, f1-score of 96.86%, and false alarm rate of 00.06% for the CICIoT 2023 dataset and an accuracy of 95.55%, precision of 88.78%, detection rate of 57.77%, Matthew's correlation coefficient (MCC) of 79.26%, f1-score of 70.06%, and false alarm rate of 00.81% for the UNSW_NB15 dataset. These findings reveal that the proposed model is proficient in identifying cyberattacks and exhibits adaptability in detecting a multitude of cyberattacks within real IIoT environments.

Keywords Industrial Internet of things · Capsule network · Machine learning · Deep learning

1 Introduction

Industrial Internet of Things (IIoT) extends industrial IoT applications. Industrial activities have become more reliable and efficient with the IIoT. By incorporating a range of cyber-physical systems and modern communication technologies, smart manufacturing systems increase industrial system monitoring and control [1]. Smart manufacturing is crucial to understanding Industry 4.0, the next industrial revolution. Many sensors, actuators, and sophisticated technologies have applications in the industry. A recent study found over 8 billion IoT-connected devices, with 41 billion by 2027. With industries including "automotive, smart home, manufacturing, energy, healthcare, transportation, logistics, and media at the forefront of IoT advancement, the global IoT market surpassed $380 billion in 2021 and is projected to exceed $1.8 trillion by 2028, with a compound annual growth rate (CAGR) of 25.4% [2]".

The industry now values dependability, response time, and network latency. Given these considerations, optimizing data transmission and decision-making technology necessitates automation, eliminating human intervention. The IoT has become one of the most promising research

✉ Qawsar Gulzar
qawsar2008854@st.jmi.ac.in

Khuram Mustafa
kmustafa@jmi.ac.in

[1] Department of Computer Science, Jamia Millia Islamia, Jamia Nagar, Okhla 110025, New Delhi, India

Published online: 24 January 2025

fields in recent decades, interconnecting infinite consumer gadgets to improve customers' everyday lives [3]. Industry 4.0 envisions using the IoT to boost industrial productivity, efficiency, and security [4]. The IIoT focuses on industrial IoT efficiency.

Furthermore, given the insufficient security measures observed in many cases, it is imperative to consider other vital factors in IIoT, including reliability, scalability, and energy consumption [5]. In the first six months of 2021, "DarkSide, a Russian hacker collectively, asserted accountability for a ransomware assault targeting Colonial Pipeline, a prominent fuel pipeline in the United States". Consequently, the company was compelled to disable its SCADA systems and remit approximately $5 million in a difficult-to-retrace cryptocurrency.

In pursuit of this objective, multiple researchers have put out diverse models to identify and alleviate the effects of such attacks, such as network intrusion detection systems (NIDS) [6].

1.1 Motivation

The extensive utilization of IoT devices frequently results in a substantial rise in network traffic. Hence, distinguishing between a typical surge in traffic and an atypical surge, including several cyberattacks, might be challenging. The detection of adverse attacks necessitates the presence of intelligence to tackle such problems effectively.

Machine learning (ML) and deep learning (DL) can predict various situations to defend against attacks on IIoT networks. This phenomenon gives rise to novel business opportunities and amplifies the need for automation technologies and enhanced application development. Numerous studies have provided substantial evidence supporting the efficacy and efficiency of ML and DL algorithms for improving the privacy and safety of Industrial Internet of Things (IIoT) networks.

Inspired by prior research in intrusion detection, our study aimed to create a hybrid learning model by integrating a capsule network with deepCLG to improve the efficacy of intrusion detection using network-driven data. This paper presents unique contributions in the following areas:

1. We utilized the Boruta feature selection technique, which leverages a random forest classifier to extract the most significant features based on their relevance scores.
2. We introduced a deepCLG model comprising convolutional neural networks (CNN), long short-term memory (LSTM), and gated recurrent unit (GRU) models. This approach leverages the benefits of each strategy to enhance detection, precision, and response times to cyber attacks within IIoT settings.
3. We verified our model using extensive datasets from genuine IIoT activities, confirming that it can withstand and thrive in various cyberattacks.

The subsequent sections are structured as follows: Sect. 2 overviews current IoT and IIoT security research. Section 3 encompasses the preparation process and the recommended methodology for the system. Section 4 elaborates on the proposed approach's theoretical, mathematical, and architectural aspects. Section 5 outlines the dataset selection software, hardware, and result analysis, along with state-of-the-art comparisons and potential merits and demerits of the proposed approach. The final portion functions as the study's conclusion.

2 Existing work

The extensive application of deep learning across various domains of Big Data has sparked considerable interest in cybersecurity, particularly regarding its reliability and security. Deep learning algorithms may be classified based on their architectural design, including generative and discriminative approaches [7]. As mentioned above, the algorithms show significant promise in effectively tackling the security concerns encountered in IIoT systems [8]. This paper discusses recent research on deep learning-based security approaches for the Internet of Things (IoT) and the Industrial Internet of Things (IIoT). In this part, we do an in-depth evaluation of the best research on DL for cyberattack detection in the IoT and IIoT settings.

The solution for distributed attack detection in IoT networks, harnesses the power of deep learning techniques. The authors' innovative method involves deploying cyber-attack detection and learning models within the fog computing layer. The performance assessment incorporates three cyber security datasets: NSL-KDD, ISCX, and KDDCUP99, though limited to two models. Their findings, a testament to the cutting-edge nature of this research, indicate 71%, 98.56%, and 97% accuracy rates for R2L.U2R attacks, Probe attacks, and DoS attacks, respectively [7]. Another groundbreaking development, the anomaly detection system (ADS), was introduced by [10] to detect cyber-attacks within the IIoT. The ADS structure employs advanced DL techniques, specifically utilizing the unsupervised deep auto-encoder technique to analyze traditional network layouts and offer precise configurations. Performance is assessed using both NSL-KDD and UNSW-NB15 datasets [7]. The experimental outcomes, a testament to the effectiveness of this innovative approach, indicate a TPR of 99% and an FPR of 1.8% [7]. The proposed approach lacks real-world implementation.

In [11] the authors proposed an intrusion detection technique to mitigate botnet assaults while analyzing DNS

functionalities within IoT-enabled smart city applications. The suggested technique utilizes a dual-tier setup for monitoring DNS records and doing domain name examinations and searches using deep learning algorithms developed by the domain creation algorithm. The objective is to reduce false alarm rates.

The suggested architecture in [12] entails integrating two deep learning methods, a Distributed CNN scheme and an LSTM architecture, to identify phishing and Botnet attacks and identify adversaries attempting to enter irrelevant data. An IoT micro-security extension leverages the DCNN, whereas the backend server utilizes the LSTM. The performance assessment utilizes the N-BaIoT dataset, demonstrating 98%, and 94.30% accuracy throughout the training and testing phases, but lacks emerging attack detection.

In their study, [14] introduced a network forensics framework, PDF, to identify and track attack patterns inside the IoT networks. The PDF technique has three distinct stages: (1) data extraction, (2) deep learning parameter adaptation, and (3) anomaly incidence identification. The second step employs the particle swarm optimization, while the third employs the deep neural model. The empirical findings demonstrate a precision rate of 99.9% in contrast to 93.2% achieved by DT and 72.7% achieved by NB whereas the deployment of the approach in the other IoT networks is missing [7].

Subsequently, [13] developed an intrusion detection method using the deep belief network approach to identify intrusions in the internet of medical things (IMoT). The effectiveness of the proposed mechanism was assessed utilizing the CICIDS 2017 dataset, yielding accuracy rates of 97.71% for PortScan assaults and 96.37% for infiltration attacks. The approach lacks the classification of the attacks present in the dataset.

The authors in [7] employed deep learning and transfer learning concepts to enhance cybersecurity in IoT networks. This study introduces a personalized deep-learning algorithm for embedding layers within the Bot-IoT dataset but is restricted to lightweight datasets. Additionally, researchers in [15] utilized a hybrid approach incorporating the spider monkey optimization (SMO) technique and the stacked-deep polynomial network (SDPN) to optimize detection and identification and attain optimal outcomes. The approach lacks complete dataset implementation. To identify intrusions in IIoT designs, the researchers in [16] presented three distinct neural networks: CNN, CNN+LSTM, and another hybrid of CNN and LSTM. In the UNSW-NB15 dataset, the CNN+LSTM hybrid model obtained 93.1% accuracy for binary classification and 92.9% accuracy for multi-classification; in the X-IIoTID dataset, the hybrid model scored 99.84% accuracy for binary classification and 99.80% accuracy for multi-class classification.

The authors in [17] presented a novel method of preprocessing data for a lightweight intrusion detection system (IDS) that uses deep learning and machine learning classifiers. The datasets used for the testing and assessment in this study were the BOT-IoT and TON_IoT Network datasets. In their study, the authors introduced a lightweight intrusion detection system to safeguard IoT networks against distributed denial of service (DDoS) attacks. The system achieved an overall accuracy rate of 99%. Implementation on the network side or at the fog layer is missing besides Software Defined Networks.

The authors in [18] proposed a novel IDS, federated-SRUs, to protect IoT-driven ICSs. The model enhances SRUs to lower computing costs, solve gradient vanishing, and achieve 99.89 % accuracy on the gas pipeline dataset. It also optimizes local–global server communication via asynchronous federated learning and needs evaluation on diverse datasets. The proposed hybrid model in [19] for intrusion detection in federated learning setup attained 99.99% accuracy for CICIDS 2017 and EdgeIoTset datasets but lacks real-world application. The model lacks Implementation in the real-world setup. The important points of these works are summarized in Table 1.

Many researchers have focused on developing threat detection systems for IoT and IIoT applications. These models were predominantly evaluated using the open-source NSL-KDD and KDD Cup 99 datasets, extensively utilized and curated explicitly for Industrial Internet of Things (IIoT) applications. Most of the researchers ignored the concept of dataset balancing which otherwise leads to bias in cyberattack detection due to a lack of marginalised cyberattack class representation in the IIoT datasets. Therefore, to adhere to the current security requirements of IIoT networks, it is essential to have the new datasets. Another limitation of the groundbreaking study is the lack of attention given by most researchers to adapting their proposed models for devices with limited resources. The primary goal of this study is to address these concerns by introducing an innovative deep learning-based methodology for identifying computer attacks. This technique employs a newly created dataset targeting security in Industrial Internet of Things (IIoT) devices. Here, the Boruta feature selection criteria utilizes the supplied feature vector to locate important features. Subsequently, feature importance and impurity are utilized in the analysis. The feature drop-out strategy is employed to remove features with lower significance. This paper presents a novel and effective DeepCLG hybrid learning methodology to predict attacks in Industrial Internet of Things (IIoT) networks. This model incorporates the up-to-date and extensive CICIoT 2023 and UNSW_NB15 datasets, which focus on identifying cyber-attack patterns.

Table 1 Summary of the existing work

Index	Year	Application area	Basic idea	Feature Selection	Classifier	Dataset employed
[9]	2018	Social internet of things	Distributed attack detection system at the fog computing layer	NA	Deep learning approach	NSL-KDD, ISCX, and KDDCUP9 datasets
[10]	2018	Industrial internet of things	Identification of malicious activities in IIoT	Deep Sparse Auto-encoder Algorithm	Supervised DNN	NSL-KDD and UNSWNB15 datasets
[11]	2020	Internet of Things	Botnet detection system based on deep learning for smart cities	NA	Deep Learning Architectures	DS1-D1 and DS1-P1 datasets
[12]	2020	Internet of Things	Distributed deep learning for threat intelligence	CNN embeddings	CNN and LSTM	N-BaIoT dataset
[13]	2020	Internet of Medical Things	Intrusion detection using a Deep belief NN	Greedy layer-wise method	Deep belief neural network technique	CICIDS 2017 dataset
[14]	2020	Internet of things	A network forensic framework for attack detection in IIoT networks	Correlation coefficient	DNN	Bot-IoT and UNSW_NB15 datasets
[7]	2021	Agriculture 4.0	DDoS attack detection in Agriculture 4.0	NA	CNN, DNN, and RNN	CIC-DDoS 2019 and TON_IoT datasets
[15]	2022	Industrial Internet of Things	Deep learning-based IDS in securing IoT	SMO(sequential minimal optimization) Algorithm	stacked-deep polynomial network (SDPN)	NA
[16]	2023	Industrial Internet of Things	A hybrid learning system for intrusion detection for IIoT networks	NA	CNN, LSTM, and CNN+LSTM	UNSW-NB15 and X-IIoTID datasets
[17]	2023	Industrial Internet of Things	DDoS attack detection in lightweight IoT networks	Extra Tree Classifier	ANN, LSTM, NB, LSVM	BoT-IoT and, TON_IoT datasets
[18]	2023	Industrial Control Systems	A federated simple recurrent units for cyberattack detection against IoT-Augmented ICS	ICA	Federated RNN	Gas pipeline dataset
[19]	2024	Internet of Things	A federated learning-based deep hybrid learning model for intrusion identification in IoT	NA	CNN, Bi-LSTM	CICIDS 2017 and EdgeIoTset datasets

3 Proposed methodology

The attack detection architecture comprises many operations. The process for detecting attacks is shown in Fig. 1 and the architectural details of the model are shown in Fig. 2. The first stage of this architectural system involves collecting and observing the dataset. The dataset was gathered and examined during this phase based on its data type. Following that, the dataset underwent preprocessing, encompassing data purification, visualization, synthesis of features, and vectorization. Such techniques were employed to acquire data attributes, which were then divided into feature vectors. The feature space was split into a training set of 60% a test set of 20%, and a validation set of 20% of the original data. The learning process using the proposed deepCLG hybrid learning model included using the training set. The deepCLG hybrid learning model was evaluated using a test set and employing various evaluation criteria.

3.1 Feature selection

We started by combining as many significant features from the input dataset as possible to evaluate the performance of the proposed model without running into overfitting issues. On the other hand, using the maximum features for a typical machine learning classifier increases the likelihood of overfitting. Estimation using Baruta can be employed to ascertain the significance of impurity-based features, which

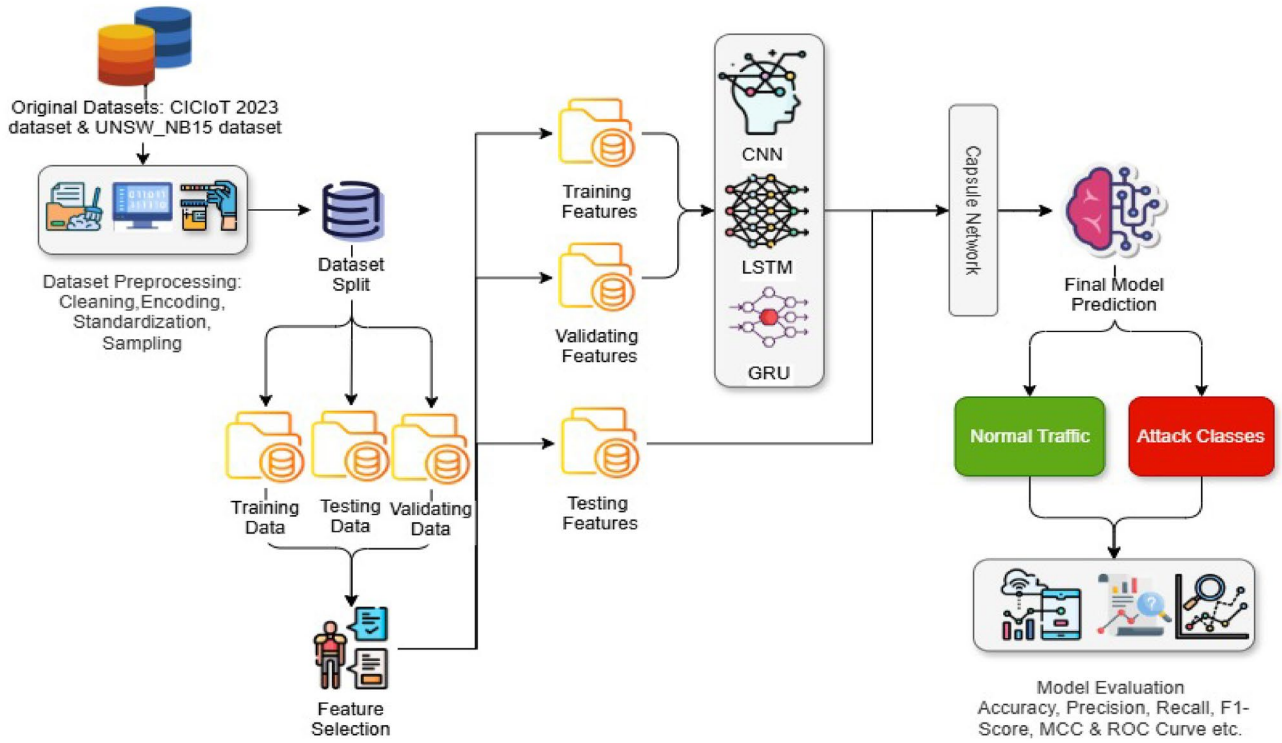


Fig. 1 Methodology of the proposed model

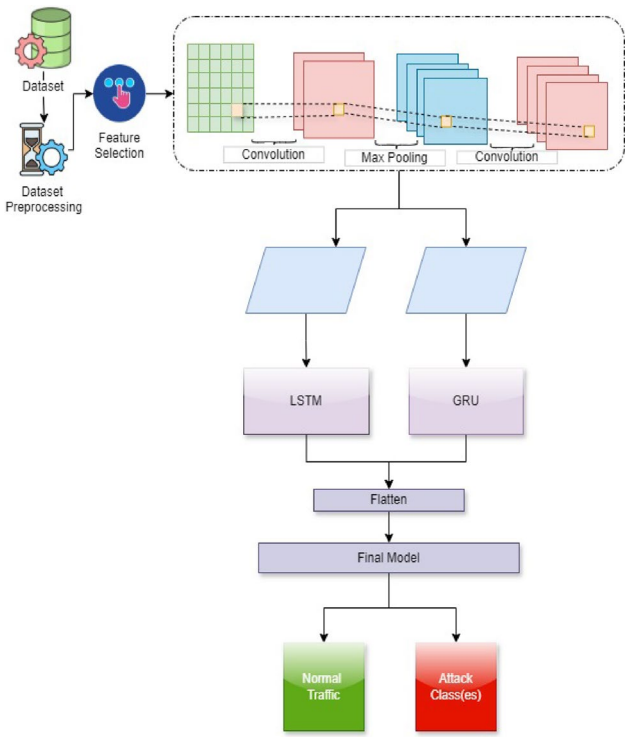


Fig. 2 Proposed CNN+LSTM+GRU architecture

can subsequently be utilized to eliminate irrelevant features based on a threshold value determined by calculating the feature importance score. We have suggested a novel feature selection called Baruta, which selects 40 features out of 47 features from the first dataset and 26 out of 41 significant features from the second dataset to satisfy the criteria of incorporating essential features into the development of the model. The remaining irrelevant features in the dataset are removed from both datasets.

The feature selection method, binary arbitration for universal targeted aggregation (BARUTA), uses random forests for feature selection and feature significance estimate. The inherent randomness in Baruta and its possible effect on feature selection results must be considered during feature rejection. Binary arbitration is used in the mathematical formulation of Baruta to choose features according to the importance scores of the random forest model. Let us dissect the Baru feature selection method step by step, including mathematical expressions as needed.

3.1.1 Random forest training

Assuming that there are m samples and n features, given the feature matrix X has dimensions $m \times n$ and the target variable y has dimensions $m \times 1$. To train a random forest (RF) model using X and y as inputs, an ensemble of decision trees $T_1, T_2, \cdots, T_k$ is constructed, where k is the forest's tree count.

At each split node, a random selection of features and a bootstrap sample of the training data are used to construct each decision tree.

3.1.2 Feature importance calculation

Calculate the importance score FI_i for each feature i when constructing the tree in RF. Consider the reduction in impurity (e.g., Gini impurity or entropy).

Gini impurity: The Gini impurity determines the class disparity within a sample collection. The calculation involves the summation of the squared probabilities associated with the selection of each class, followed by the subtraction of the value from 1. In the realm of feature selection, the decreased Gini impurity is used as a metric to ascertain the significance of a feature in the process of data splitting. The Gini impurity for a given collection of samples may be mathematically defined as:

$$I_{\text{Entropy}}(p) = -\sum_{i=1}^{c} p_i^2 \tag{1}$$

where p_i represents the probability of class I being present within the set of samples, and c denotes the overall count of classes. The calculation of the reduction in Gini impurity resulting from the splitting of feature I at node t may be determined as follows:

$$\Delta I_t(i) = I_{\text{Gini}}(p_t) - \left(\frac{n_{t_{\text{left}}}}{n_t} \cdot I_{\text{Gini}}(p_{t_{\text{left}}}) + \frac{n_{t_{\text{right}}}}{n_t} \cdot I_{\text{Gini}}(p_{t_{\text{right}}})\right) \tag{2}$$

Where p_t represents the class distribution at the node t, n_t represents the count of samples at node t, $n_{t_{left}}$ and $n_{t_{\text{right}}}$ represents the count of samples in the left and right child nodes. $p_{t_{left}}$, and $p_{t_{\text{right}}}$ represent the class distributions in the left and right child nodes, respectively.

Entropy: Entropy determines how random or unreliable a collection of samples is. The calculation involves the summation of the logarithm of the products of the probabilities of each class. Identical to the Gini impurity, the significance of a feature in separating the data is ascertained by its reduction in entropy. Mathematically, entropy for a collection of samples is computed as:

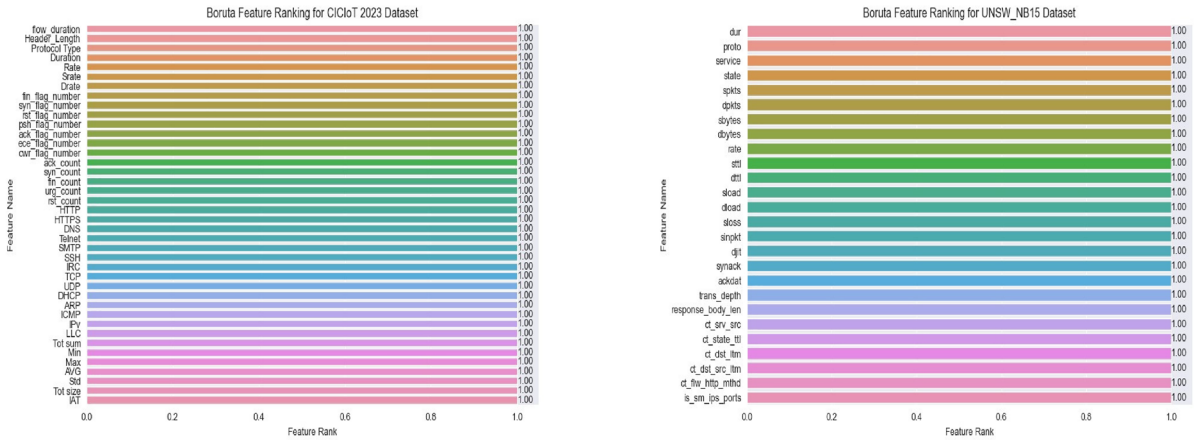
$$I_{\text{Entropy}}(p) = -\sum_{i=1}^{C} p_i \log_2(p_i) \tag{3}$$

The reduction in entropy at the node t due to splitting on feature I can be calculated similar to the Gini impurity:

$$\Delta I_t(i) = I_{\text{Entropy}}(p_t) - \left(\frac{n_{t_{\text{left}}}}{n_t} \cdot I_{\text{Entropy}}(p_{t_{\text{left}}}) + \frac{n_{t_{\text{right}}}}{n_t} \cdot I_{\text{Entropy}}(p_{t_{\text{right}}})\right) \tag{4}$$

These formulas not only provide the mathematical basis for calculating entropy and Gini impurity but also serve as the foundation for determining feature importance scores in decision trees or random forests. Understanding these scores is crucial as they can guide in making informed decisions. These scores are calculated and demonstrated in Fig. 3 below for the two datasets, showcasing their practical relevance. Mathematically, the feature importance score FI_i for feature i can be represented as:

$$\mathrm{FI}_i = \sum_{t=1}^{k} \frac{n_t}{n} \cdot \Delta I_t(i) \tag{5}$$



(a) Feature importance score for CICIoT 2023 dataset

(b) Feature importance score for UNSW_NB15 dataset

Fig. 3 Feature importance scores for two datasets

Where n_t is the collection of samples at the node t, n is the total number of samples, and $\Delta I_t(i)$ is the reduction in impurity/ entropy due to the feature I at node t.

3.1.3 Binary arbitration

Following the computation of feature importance scores, a binary arbitration process chooses the features to be preserved for further analysis. A threshold Thresh is specified for features that have importance scores higher than this threshold are chosen. Mathematically, the binary arbitration process can be represented as:

$$\text{Selected}_{\,i} = \begin{cases} 1 & \text{if } \text{FI}_{\text{i}} > \text{Thresh} \\ 0 & \text{otherwise} \end{cases}. \tag{6}$$

3.1.4 Selected feature subset

Ultimately, the selected feature subset, represented by the letter S is produced by selecting the features for which $\text{Selected}_{\,\text{i}} = 1$. In mathematical terms, features I for which $_{\text{i}} = 1$ make up the selected feature subset S. These steps, which include training the model, calculating feature importance scores, carrying out binary arbitration, and choosing the final feature subset, provide a mathematical foundation for comprehending how Baruta uses random forests for feature selection. In addition, the pseudocode for the Boruta feature selection technique is stated in Algorithm 1.

4 Mathematical description and architecture behind the proposed model

This research aims to enhance the classification accuracy of cyber-attacks in IIoT networks by using the DeepCLG Hybrid learning model in conjunction with the Capsule network. The proposed approach focuses on the alleged network traffic behaviour to detect network traffic attacks. Following the proposed approach, Fig. 1 shows the overall summary of the cyberattack detection process. Upon obtaining the subset of chosen features, represented as S, produced by picking the features for which $\text{Selected}_{\,\text{i}} = 1$. The threshold for detecting variables with a significant degree of correlation is established at 0.8. This number represents the threshold for judging whether two variables are significantly correlated and worthy of removal. The objective of this code is to decrease the dimensionality of the dataset by selectively retaining the most significant features (i.e., preserving the relevant information) and eliminating features that exhibit strong correlations. These mathematical tenets and parameters guide feature selection methods and multicollinearity reduction, eventually improving the reliability and efficiency of the dataset for application in DL models. The features are generated from both the training and test datasets to validate the model.

Algorithm 1 Boruta_Feature_Selection

```
1. Normalization (X, y)
2. ROS = RandomOverSampler(sampling_strategy, random_state)
3. X_resampled, y_resampled = ROS.fit_resample(X_scaled, y)
4. Input: X_train, y_train
5. RF= RandomForestClassifier(n_estimators, n_jobs, class_weight, max_depth)
6. Boruta_selector = BorutaPy(RF, n_estimators, verbose, random_state,max_iter)
7. TruncatedSVD(n_componenets)
8. Svd.fit_transform(X_train)
9. X_train_dense = X_train_Svd
10. Boruta_selector.fit ( X_train_dense, y_train[:,0])
11. Feature_ranks = Boruta_selector.ranking_
12. Selected_feature_indices = np.where(Boruta_selector.support_)[0]
13. Best_features = array(data.columns)[:,-1][Selected_feature_indices]
# Drop highly correlated variables
14. Correlation_matrix = DataFrame(X_train, columns=data.columns[:-1])[best_features].corr()
15.Upper_triangle=Correlation_matrix.where(np.triu(np.ones(Correlation_matrix.shape),k=1).astype(np.bool))
16. To_drop = [Column for Column in Upper_triangle.columns if any(Upper_triangle[column] > 0.8)]
17. X_train_filtered = DataFrame(X_train, Columns=data.columns[:-1])[Best_features].drop(To_drop, axis=1)
18. X_test_filtered = DataFrame(X_test, Columns=data.columns[:-1])[Best_features].drop(To_drop, axis=1)
```

4.1 Deep learning models

Subsequently, the input layers for CNN, GRU, and LSTM models receive input samples that are one-dimensional arrays with a length equivalent to the number of features after filtering out highly correlated features. Each feature is considered a single channel in sequential or time series data, whereas, in image data, it is treated as multiple channels. Subsequently, the outputs are categorized into two distinct classes: regular traffic class and attack traffic class. The way we tuned our deepCLG hybrid learning models is shown in Table 2 as follows:

4.2 Mathematical description behind the capsule network and the deep learning models.

4.2.1 Capsule network (CapsNet)

Sequential data processing models such as CNNs, GRUs, and LSTMs are integrated with Capsule Networks. This method is especially beneficial for tasks that include sequential data, where capturing spatial hierarchies over time with the aid of Capsule Networks via dynamic routing [20] is crucial. The following discussion will explore the operational mechanism of the neural capsule seen in Fig. 4 and the

Table 2 Hyperparameter tunning inside the CNN_LSTM_GRU model

Parameter(s)	Value(s)
Language	Python
Libraries	Pandas, Numpy, Matplotlib, Sckitlearn, Seaborn, Matplotlib, Keras, Imblearn, and Tensorflow
Train set	60%
Test set	20%
Validation set	20%
Conv1D filters	[64,32]
Conv1D kernel size	[5,3]
LSTM units	[64,32]
GRU units	[32,64]
Dropout	[0.2-−0.3]
Dense units	[128,1]
Capsule layer	[num_capsule=10, dim_capsule=16, routings=3]
Activation function	[Relu, Softmax]
Loss function	Categorical cross-entropy
Optimizer	Adam
Learning rate	0.1 – 0.001
Dropout rate	0.2 – 0.3
Number of Epochs	50
Batch size	64

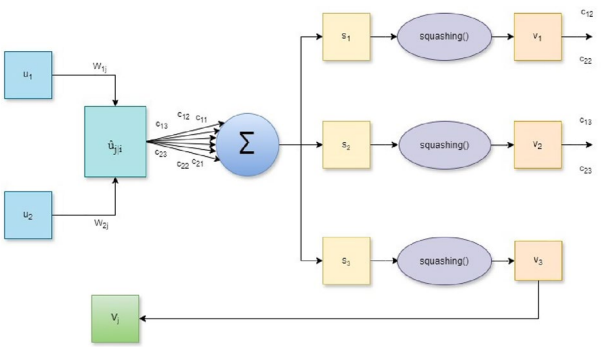


Fig. 4 Capsule network

mathematical aspects behind integrating the Capsule Layer with sequential data models. This integration may be concisely stated through four steps: matrix transformation, input weighting, summing up the weights, and applying nonlinear transformation [21]. The neural network consists of multiple capsule layers, each structured into numerous collections of vector neurons. This arrangement ensures that every neuron can sense both length and direction.

Figure 4 shows how the transformation matrices W_{1j}, W_{2j} and W_{3j} turn the underlying capsules u_1, u_2 and u_3 to a predicted vector $\hat{u}_{j|i}$:

$$\hat{u}_{j|i} = u_i \cdot W_{i|j} \tag{7}$$

Once the value of $\hat{u}_{j|i}$, has been determined, the second step involves assigning weights to compute the node s_j, across the iterations denoted by the variable r:

$$s_j = \sum_i c_{ij} \cdot \hat{u}_{j|i} \tag{8}$$

where, c_{ij} represents the coupling coefficient, which is dynamically adjusted through the dynamic routing method, while b_{ij} of the SoftMax function starts at 0, k denotes the number of neurons, and the coupling coefficient c_{ij} is determined as follows:"

$$c_{ij} = \frac{\exp\left(b_{ij}\right)}{\Sigma_k \exp\left(b_{ik}\right)} \tag{9}$$

The Capsule network utilizes the Squash activation function to produce the output, represented as v_j, which is computed as follows:

$$v_j = \frac{\left\|s_j\right\|^2}{\left\|1 + s_j\right\|^2} \cdot \frac{s_j}{\left\|s_j\right\|} \tag{10}$$

The first part of this activation function scales the input vector s_j, while the second part normalizes it to a unit vector s_j. This process preserves the input vector's direction and compresses its magnitude to a range of (0,1), which helps prevent gradient explosion [21]. The variable b_{ij} is updated by evaluating the consistency between the output v_j of the current layer and the prior layer's estimated output $\hat{u}_{j|i}$.

$$b_{ij} \leftarrow b_{ij} + \widehat{u}_{j|i} \cdot v_j \tag{11}$$

The coupling coefficient c_{ij} is dynamically updated through routing mechanisms across the network, establishing a pathway with the sought-optimal parameter being the pathway's terminus.

Algorithm 2 Routing Algorithm

1. **procedure** Routing $\left(\widehat{\boldsymbol{u}}_{j|i}, \boldsymbol{r}, \boldsymbol{l}\right)$ |
2. for all capsule i in layer l and capsule j in layer $(l+1)$: $\boldsymbol{b}_{ij} \leftarrow \mathbf{0}$
3. **for** r iterations **do**
4. for all capsule i in layer l : $\boldsymbol{c} \leftarrow \mathbf{softmax}\left(\boldsymbol{b_i}\right)$ ▷ softmax computes Eq. 9
5. for all capsule j in layer $(l+1)$: $\boldsymbol{s}_j \leftarrow \sum_i \boldsymbol{c}_{ij} \cdot \widehat{\boldsymbol{u}}_{j|i}$
6. for all capsule j in layer $(l+1)$: $\boldsymbol{v}_j \leftarrow \boldsymbol{squash}\left(\boldsymbol{s}_j\right)$ ▷ squash computes Eq. 10
7. for all capsule i in layer l and capsule j in layer $(l+1)$: $\boldsymbol{b}_{ij} \leftarrow \boldsymbol{b}_{ij} + \widehat{\boldsymbol{u}}_{j|i} \cdot \boldsymbol{v}_j$
8. **return** $\mathbf{v_j}$

4.2.2 Deep learning network

Fig. 5 shows the architecture of the proposed model for cyberattack detection in IIoT networks in our paper. Its models include deep GRU, deep LSTM, and deep CNN. The one-dimensional convolutional neural network (1D CNN) is extensively used to analyse sequential data because it can effectively capture local patterns, hierarchical features, and translation invariance. These properties make 1D CNNs an effective method for extracting significant information from sequential data.

The Deep CNN model has three layers. The first layer consists of a convolutional layer with 64 filters and 5 kernels. Subsequently, a maxpooling layer is applied with a pool size of 2. The third layer comprises a convolutional layer with three kernels and thirty-two units. The repeatvector layer follows this one. We used a dropout regularization rate of 0.2 between the second and third layers to help the model overcome the overfitting issue.

The following expressions determine the CNN model's outputs y_t^{CNN} and hidden layers h_t^{CNN} for a given selected feature vector $\left\{x_1^{CNN}, x_2^{CNN}, \ldots x_t^{CNN}\right\}$:

$$h_t^{CNN} = \sigma\left(w_i x_t^{CNN} + w_R h_{t-1}^{CNN} + b_h\right) \tag{12}$$

$$y_t^{CNN} = \sigma\left(w_y h_t^{CNN} + b_y\right) \tag{13}$$

The relationships between a neural network's input, output, and hidden layers are represented by the weighted matrices w_i, w_R, and w_y. Additional parameters, namely bias vectors b_h and b_y, modify the network's output. The symbol t" represents the number of samples, whereas "r" denotes the selected activation function. The Rectified Linear Unit (ReLU) is one activation function that could be considered.

There are three layers in the LSTM model. It effectively addresses the issue of vanishing gradient. The initial layer comprises 64 units, while the subsequent layer comprises 32. A dropout rate of 0.3 is applied between these two layers.

The Long Short-Term Memory (LSTM) model consists of three multiplication components: the input gate i_t^{LSTM}, forget gate f_t^{LSTM}, and output gate o_t^{LSTM}. The regulation of the cell c_t^{LSTM}, these gates significantly influence which is in charge of storing both previous and future information.

The following calculation determines the components:

$$f_t^{LSTM} = \sigma\left(w_f\left[h_{t-1}^{LSTM}, x_t^{LSTM}\right] + b_f\right) \tag{14}$$

$$i_t^{LSTM} = \sigma\left(w_i\left[h_{t-1}^{LSTM}, x_t^{LSTM}\right] + b_i\right) \tag{15}$$

$$\tilde{c}_t^{LSTM} = \tanh\left(w_c\left[h_{t-1}^{LSTM}, x_t^{LSTM}\right] + b_c\right) \tag{16}$$

$$c_t^{LSTM} = f_t^{LSTM} * c_{t-1}^{LSTM} + i_t^{LSTM} * \tilde{c}_t^{LSTM} \tag{17}$$

$$o_t^{LSTM} = \sigma\left(w_0\left[h_{t-1}^{LSTM}, x_t^{LSTM}\right] + b_0\right) \tag{18}$$

$$y_t^{LSTM} = o_{t^*}^{LSTM} \tanh\left(c_t^{LSTM}\right) \tag{19}$$

where the variable i denotes an "input gate", f denotes a "forget gate", o denotes an "output gate", and c denotes a "cell activation [22]". The matrices representing weights are

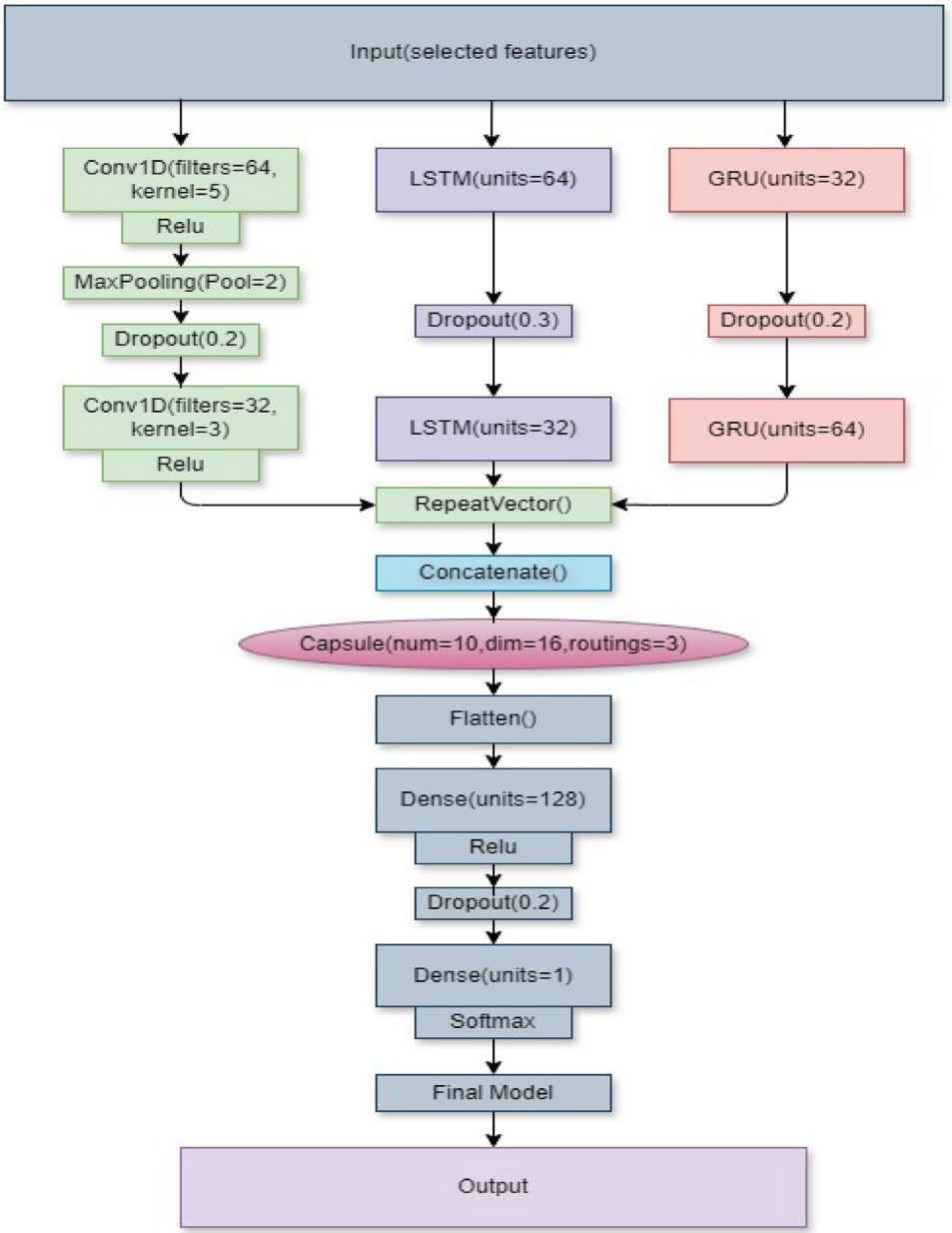


Fig. 5 Overall architecture of the proposed model for cyber-attack detection in IIoT environments

commonly called w_i, w_f, w_o, and w_c. The selected feature vector is denoted as $\{x_1^{LSTM}, x_2^{LSTM}, \ldots x_t^{LSTM}\}$, the output of LSTM is denoted as y_t^{LSTM}, the previous hidden states are denoted as h_{t-1}^{LSTM}, and the bias vectors are represented as b_i, b_f, b_c, and b_0.

The deep GRU model consists of three layers, it effectively addresses the issue of vanishing gradient [22]. The first layer consists of 32 units, whereas the following layer consists of 64 units [22]. A dropout rate of 0.2 is applied between these two layers.

The components are ascertained through the subsequent computation:

$$z_t^{GRU} = \sigma\left(w_z\left[h_{t-1}^{GRU}, x_t^{GRU}\right]\right) \tag{20}$$

$$r_t^{GRU} = \sigma\left(w_r\left[h_{t-1}^{GRU}, x_t^{GRU}\right]\right) \tag{21}$$

$$y_t^{GRU} = \left(1 - z_t^{GRU}\right) * h_{t-1}^{GRU} + z_t^{GRU} * \tanh\left(w\left[r_t^{GRU} * h_{t-1}^{GRU}, x_t^{GRU}\right]\right) \tag{22}$$

Where Z_t^{GRU} is an "update gate", y_t^{GRU} is an "output" of GRU, y_t^{GRU} is a "reset gate", $\{x_1^{GRU}, x_2^{GRU}, \ldots x_t^{GRU}\}$ is a selected feature vector, h_{t-1}^{GRU} are "hidden states" and W_z, W_r, and w are the weight matrices [22]. In contrast to LSTM, it lacks storage modules [22]. The output sizes of deep

LSTM and deep GRU are identical [22], leading to similar computations.

Subsequently, the outputs of the deep LSTM and deep GRU models are transformed into dimensions that align with those of the deep CNN model by utilising the RepeatVector() function. Next, the outputs of all these models were combined to align with the temporal dimensions. Subsequently, the concatenate function is employed to merge the outputs of the three deepCLG models. A robust DeepCLG hybrid learning model is constructed by passing the concatenated merged output as an input to the capsule layer. The composition of the capsule layer includes input dimensions of 10, output dimensions of 16, and routings of 3.

After the capsule layer, a single flattened layer is observed. The flattened layer's output is used as input for the dense layers. The first dense layer consists of 128 units and is activated using the Relu function. The second dense layer has 1 unit, followed by a softmax activation function. Lastly, the final model with a learning rate of 0.001 and the early stopping with a patience value of 15 predicts multiple outcomes, as shown in Fig. 5 shows the overall architecture of the proposed model. In addition, the pseudocode for the CNN + LSTM + GRU model is stated in Algorithm 3. The total computational complexity of your model is determined by the sum of the complexities of all its layers. Here is a breakdown of how everything is connected:

$$T(n) = O\left(n \times \max\left(f_1 \times k_1, f_2 \times k_2, U^2_{GRU}, U^2_{LSTM}, d \times d_1\right) + c \times d \times r\right) \tag{23}$$

where: n represents input size, f_1,f_2 depicts filter count in CNN layers, k_1, k_2 represents kernel sizes in the CNN layers, $u^2_{GRU} + u^2_{LSTM}$ represents the count of units in the GRU and LSTM layers, and c, d, r represents the count of capsules, dimensions and routings iteration in the Capsule layer.

This complexity suggests that the model's performance will increase linearly with the input size n, while simultaneously contingent upon different layers' arrangement.

Algorithm 3 CNN_LSTM_GRU

```
1. Cnn_input  = input(shape=(X_train_filtered))
2. Lstm_input =  input(shape=(X_train_filtered))
3. Gru_input =  input(shape=(X_train_filtered))
4. Cnn_model = Sequential([Conv1D(filters,kernel_size,  activation),Maxpooling1D(pool_size),dropout(rate),
Conv1D(filters,kernel_size,activation)])
5. Lstm_model = Sequential([Lstm(units,return_sequence), dropout(rate), dense(layers)])
6. Gru_model = Sequential([Gru(units,return_sequence), dropout(rate), dense(layers)])
7. Cnn_output = Cnn_model(Cnn_input)
8. Lstm_output =RepeatVector(Cnn_output)(Lstm_model(Lstm_input))
9. Gru_output =RepeatVector(Cnn_output)(Gru_model(Gru_input))
10. Merged_output = Concatenate(axis([Cnn_output,Lstm_output,Gru_output]))
11. Capsule_layer = CapsuleLayer(num_capsule, dim_capsule, routings)(Merged_output)
12. Output = Flatten()(Capsule_layer)
13. Output = Dense(units, activation)(Output)
14. Output =Dropout(rate)(Output)
15. Output = Dense(y_train, activation)(Output)
16. Final_model = Model(inputs=[Cnn_input, Lstm_input, Gru_input], Outputs=Output)
17. Final_model = Compile(optimizer, loss, metrics = ['accuracy', Precision(), Recall()])
18. Def Schedular(epoch, lr)
19. Callback=LearningRateScheduler(scheduler)
20. Early_stopping(monitor, patience, restore_best_weights)
21.Final_model.fit([X_train_filtered,X_train_filtered,X_train_filtered],y_train, callbacks[Early_stopping,Callback],
epochs, batch_size, verbose)
```

4.3 Simulation environment

As illustrated in Fig. 5, the DeepCLG hybrid learning model is the suggested method for attack detection in IIoT systems in this study. The configuration of this model comprises three input layers, nine hidden layers, and one output layer. The CICIoT 2023 dataset and UNSW_NB15 datasets were acquired to implement this model. The dataset known as CICIoT 2023 comprises a comprehensive set of 47 features. Hence, in the feature selection process, it was observed that 40 out of the total 47 features exhibited a feature importance score of 1.0 for the CICIoT 2023 dataset. Consequently, the random forest classifier rejected the remaining features. Similarly, the random forest classifier selected 26 of 41 features for the UNSW_NB15 dataset. The input features for the two datasets were designated as $x_1, x_2 \ldots, x_n$ feature vector sets. Each dataset is split into 60%, 20%, and 20% slices, respectively. The input layers assign weights and bias values, which are then transmitted to the hidden layers along with the appropriate dropout rates of (0.2 and 0.3) to facilitate subsequent processing. The learning process is crucial in hidden layers as it is pivotal in accurately predicting the output based on actual features. The information is transferred from the hidden layers to the output layer to generate appropriate output. Once the trained model has been acquired, it forecasts attacks using a test set.

4.4 Evaluation parameters

A range of evaluation metrics were used to monitor the efficacy of the proposed deepCLG hybrid learning model. The subsequent section briefly describes the performance parameters for evaluating the proposed model.

1, Accuracy: Accuracy is defined as the proportion of instances that were accurately anticipated for all instances:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{24}$$

2, Precision (PPV): Precision denotes the proportion of events anticipated to be positive by the model that are actually positive and is spelt as:

$$\text{PPV} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{25}$$

3, Recall (DR) / Sensitivity (TPR): Recall, sometimes referred to as sensitivity or true positive rate, is the ratio of real positives accurately detected by the model:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{26}$$

4, Specificity: The classifiers 'True Negative Rate (TNR)':

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{27}$$

5, False Positive Rate (FPR): It assesses the probability that a model will erroneously classify any negative event as positive.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{28}$$

6, False Negative Rate (FNR): A ratio of mistakenly anticipated negative outcomes out of the actual positive outcomes".

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} \tag{29}$$

7, F1 - Score: Represents a balance between 'precision' and 'recall', with a score of 1 showing flawless accuracy and recall:

$$\text{F1} - \text{Score} = \frac{2 \times \text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} \tag{30}$$

8, Receiver Operating Characteristics (ROC): The ROC AUC Score (Receiver Operating Characteristic - Area Under the Curve) quantifies a model's capacity to differentiate between classes at different threshold levels.

$$ROCAUC = \int_0^1 dt \text{TPR}\left(FPR^{-1}(t)\right) \tag{31}$$

9, Mathews Correlation Coefficient (MCC): A statistical metric known as the Matthews correlation coefficient (MCC) evaluates binary classifications' accuracy and dependability. It considers both true positives and false negatives, rendering it particularly useful in scenarios with imbalanced datasets.

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{32}$$

The MCC scale spans from -1 to +1, with 1 representing flawless prediction, 0 representing arbitrary prediction, and -1 representing complete disparity between prediction and observation.

5 Implementation and evaluation

This part offers a thorough explanation of the experimental configuration, an elaborate depiction of our experiments, a summary of our results and their discussions, and a contrast between the acquired outcomes and those outlined in other ideas.

Table 3 An attack classes list included inside the CICIoT 2023 Dataset

Attack Class	Description
Benign	Benign Traffic
DDoS	These attacks include flooding IoT operations with many threats from various sources to interrupt their accessibility
DoS	DoS assaults, like DDoS attacks, often originate from a single source
Recon	To be ready for action in the future, the adversary is trying to gather all relevant data. There are many methods to execute these attacks
Web-Based	Web-based assaults target online services using a variety of techniques
Brute Force	To gain illegal access to systems, brute-force attacks include repeatedly trying different data combinations, such as passwords or passphrases. *The objective of an attack is to systematically attempt each word provided in the dictionary to ascertain the right password.*
Spoofing	Fraudulent users may "take on the identity of a target system to intercept network communications without authorization" using spoofing attacks. This technique aims to gain illegal access to computer systems, steal data, and spread dangerous software
Mirai	A popular denial-of-service (DDoS) that targets Internet of Things (IoT) devices is the Mirai attack. In this attack, devices are compromised to build a botnet that could flood and overwhelm certain targets

Table 4 An attack classes list included inside theUNSW_NB15 Dataset

Attack Class	Description
Normal	Normal Traffic
Fuzzers	An attack seeks to uncover vulnerabilities in applications, operating systems, or networks by inundating them with a large volume of data[23] to induce malfunctions
Analysis	An attack that breaches web applications through ports, emails, and web scripts[23]
Backdoors	An attack that clandestinely obtains unauthorized access to a program or system by circumventing standard authentication procedures
DoS	It is akin to DDoS but the origin is from a single source
Exploits	Exploit attacks are cyber intrusions that leverage weaknesses in systems or software to obtain unauthorized access or inflict damage
Generic	An attack, irrespective of configurations, generates collisions for every block cipher that employs the hash function
Recon	To be ready for action in the future, the adversary is trying to gather all relevant data. There are many methods to execute these attacks
Shellcode	An attack that entails the utilization of compact segments of malicious software to exploit vulnerabilities and perform operations on a targeted system
Worms	An attack that replicates and propagates through networks exploiting security vulnerabilities

5.1 Experimental setup

The experiments were carried out utilizing a Microsoft Windows 11 Pro device equipped with an Intel(R) Core (TM) i7-10610U CPU @ 1.80GHz, operating at a frequency of 2304 MHz. The device possessed 4 cores and 8 logical processors, 32 GB RAM and a 512 GB SSD. The proposed model was evaluated using Python 3.11.4 (64-bit version), incorporating various libraries, including Numpy, Pandas, Matplotlib, Keras, Scipy, SkLearn and others.

An evaluation was conducted on the CICIoT 2023 and UNSW_NB15 datasets to determine the suggested approach's efficacy in detecting cyberattacks inside the IIoT network traffic. The model integrates the advantages of CNN, LSTM, and GRU with the BARUTA feature selection technique. The datasets are partitioned into two distinct subsets: a training set of 60%, validation set of 20%, and a test set of 20% of the original data.

5.2 Datasets description

The significance of a dataset in the testing, analysing, and evaluating a detection system must be considered. A high-quality dataset yields efficient outcomes for offline systems and holds the promise of effectiveness when used in real-world settings. We used the CICIoT 2023 benchmark dataset [24], the most recent version offered by the Canadian Institute for Cyber Security (CIC), to evaluate the efficacy

of our proposed approach. The training dataset has 708,109 records, whereas the testing dataset has 177,027 records. The dataset contains 47 features and a corresponding class label. Table 3 below outlines eight distinct general class names, consisting of one benign and seven attacks: "DDoS", "Brute Force", "Spoofing", "DoS", "Recon", "Web-based", and "Mirai."

Our proposed model is being evaluated using the renowned UNSW_NB15 dataset. This dataset was developed by the Australian Defence Force Academy (ADFA) in conjunction with the University of New South Wales (UNSW) in Canberra, Australia's School of Engineering and Information Technology, as well as the Cyber Range and IoT Labs [10]. It comprises contemporary, engineered attack actions [394,445] and represents real, modern behaviour. It contains 257,673 recordings, each with 41 characteristics and a class label (i.e., 93,000 records classified as normal and 164,673 as attacks). Table 4 below lists 10 distinct class names, one labelled as normal and nine classes representing attacks: "Fuzzers", "Analysis", "Backdoors", "DoS", "Exploits", "Generic", "Reconnaissance", "Shellcode", and "Worms".

5.3 Dataset preprocessing

5.3.1 Handling missing values

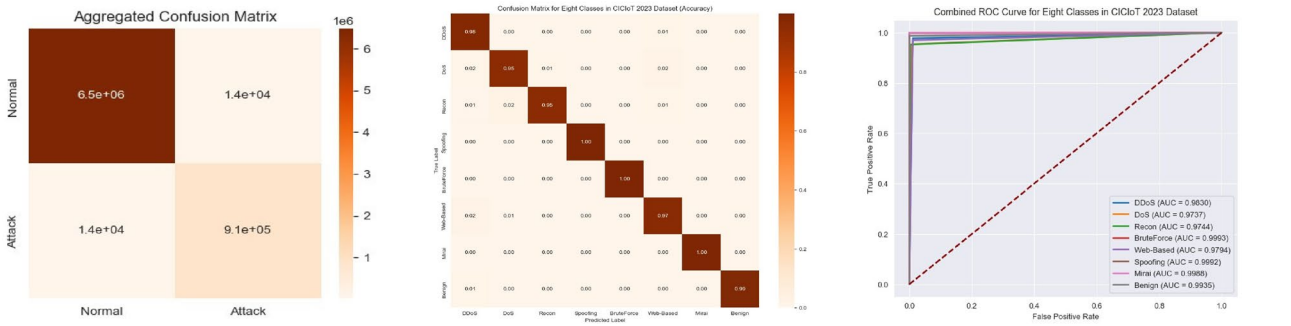
Addressing missing values is a critical and foundational step in any data pre-processing approach. Many statistical parameters, such as mean and standard deviation, are utilized in different data pre-processing procedures to substitute missing values or occurrences [17].

5.3.2 Handling categorical data

Most machine learning classifiers are not well-suited for the diversity of string values seen in numerous features across both datasets. Python has many libraries for label encoding and one-hot encoding, which use sklearn's OneHotEncoder and LabelEncoder, ultimately, to manage categorical data by replacing textual values with corresponding numeric values derived from the data classes within that particular feature [17]. One of the robust Python packages for digitizing or doing one-hot encoding/binarization of categorical data is called Scikit-learn. The suggested model uses the sklearn preprocessing tool for label encoding and one hot encoding.

Table 5 Table CICIoT 2023 Dataset 8-Class Performance

Attack Class	Precision	Recall	F1-score	TPR	TNR	FPR	FNR	Support
DDoS	0.9792	0.9772	0.9787	0.9772	0.9888	0.0111	0.0227	326,297
DoS	0.9519	0.9539	0.9529	0.9539	0.9935	0.0064	0.0460	109,872
Recon	0.9768	0.9526	0.9645	0.9526	0.9961	0.0038	0.0473	136,091
BruteForce	0.9967	0.9987	0.9977	0.9987	0.9999	0.0000	0.0012	54,618
Web-Based	0.9491	0.9698	0.9594	0.9698	0.9911	0.0088	0.0369	27,327
Spoofing	0.9993	0.9886	0.9989	0.9986	0.9889	0.0110	0.0301	163,961
Mirai	0.9933	0.9981	0.9957	0.9981	0.9993	0.0006	0.0018	82,008
Benign	0.9945	0.9871	0.9907	0.9871	0.9998	0.0001	0.0128	27,312
Total	**0.9945**	**0.9871**	**0.9907**	**0.9871**	**0.9998**	**0.0001**	**0.0128**	**927,486**



(a) Binary classification (b) Multi-class classification (c) ROC curve for Multi-class classification

Fig. 6 Confusion matrix and ROC AUC curve of CICIoT 2023 dataset for Binary and Multi-class classification

Table 6 Time Comparison of CICIoT 2023 Dataset

Neuron Count	TrainingDuration (s)	TimeSaving (%)
120	4022	97.88
256	4148	98.04
576	4218	98.96

Table 7 Time Comparison of UNSW_NB15 Dataset

Neuron Count	Training Duration (s)	TimeSaving (%)
120	2028	88.89
256	2232	94.67
576	2221	96.88

5.3.3 Data standardization

The UNSW_NB15 dataset contains several features in a format that is not model-friendly. The features consist of source and destination IP addresses in IP address format, state and protocol in categorical format, and so forth. Proper normalization of the values following label encoding is necessary [17]. The suggested approach to standardizing the input data uses Standard Scaler.

5.3.4 Avoiding class imbalance problems

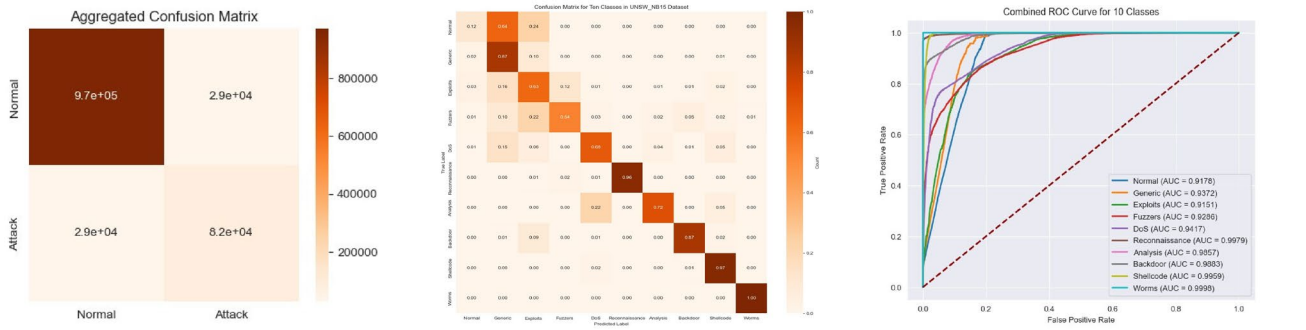
To experiment with the proposed model the CICIoT 2023 and the UNSW_NB15 datasets were balanced using the RandomOverSampler() technique. In machine learning, random oversampling is a method for rectifying class imbalance in datasets. By randomly duplicating instances until the class distribution is balanced or reaches a specified ratio, random oversampling increases the number of samples in the minority class(es).

5.4 Results and discussions

The proposed DeepCLG hybrid learning model is implemented and evaluated using Python. Both datasets were used for the tests, where the most optimal features were automatically chosen using a threshold value of 0.8, as determined by the Baruta feature selection technique. We utilized the CICIoT 2023 dataset, which consists of 4,47,000 records

Table 8 Table UNSW_NB15 Dataset 10-Class Performance

Attack Class	Precision	Recall	F1-score	TPR	TNR	FPR	FNR	Support
Worms	0.9823	1.000	0.9911	1.0000	0.9980	0.0020	0.0000	11,111
Shellcode	0.8443	0.9727	0.9039	0.9727	0.9799	0.0201	0.0273	11,203
Backdoor	0.9117	0.8683	0.8895	0.8683	0.9907	0.0093	0.1317	11,012
Analysis	0.9412	0.7208	0.8057	0.7208	0.9951	0.0049	0.0998	11,235
Reconnaissance	0.9994	0.9633	0.9810	0.9633	0.9999	0.0001	0.0367	11,137
DoS	0.7013	0.6817	0.6914	0.6817	0.9678	0.0322	0.1099	11,075
Fuzzers	0.7784	0.5400	0.6363	0.5400	0.9822	0.0178	0.1885	11,127
Exploits	0.4732	0.6465	0.5396	0.6465	0.9222	0.0778	0.1112	11,117
Generic	0.4522	0.8690	0.5877	0.8690	0.8882	0.1118	0.1310	10,999
Normal	0.6776	0.1211	0.2016	0.1211	0.9944	0.0056	0.2218	10,984
Total	**0.7654**	**0.7363**	**0.7232**	**0.7363**	**0.9708**	**0.0292**	**0.1043**	**1,11,000**

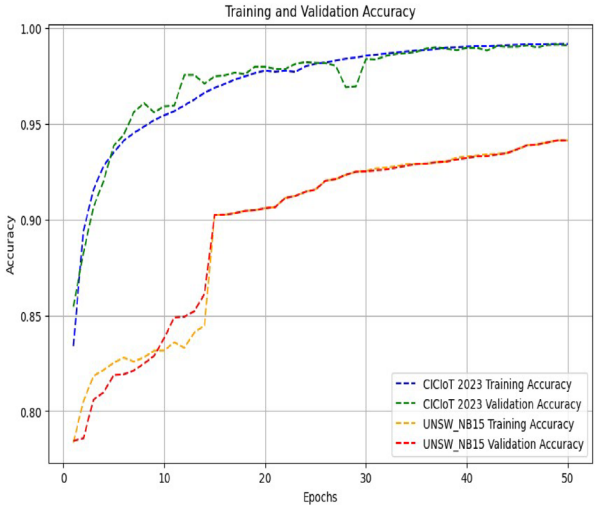


(a) Binary classification (b) Multi-class classification (c) ROC curve for Multi-class classification

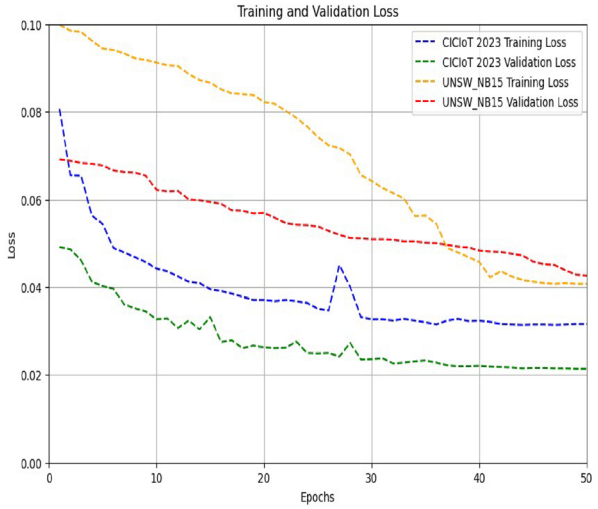
Fig. 7 Confusion matrix and ROC AUC curve of UNSW_NB15 dataset for Binary and Multi-class classification

designed for training and 89,555 records intended for testing. Similarly, the proposed DeepCLG hybrid learning model Utilized the UNSW_NB15 dataset for training and testing, including 2,57,673 records, comprising 93,000 benign records and 1,64,673 malignant records.

After analyzing the experimental outcomes, we implemented the network architecture and settings that produced the most significant detection rate (DR), the lowest false positive rate (FPR) and the false negative rate (FNR). The architecture has three input layers, nine hidden levels, and a



(a) Training and validation accuracy for CICIoT 2023 and UNSW_NB15 datasets

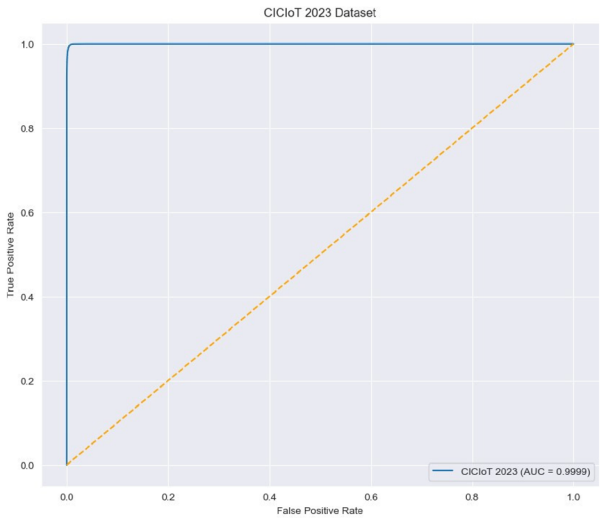


(b) Training and validation loss for CICIoT 2023 and UNSW_NB15 datasets

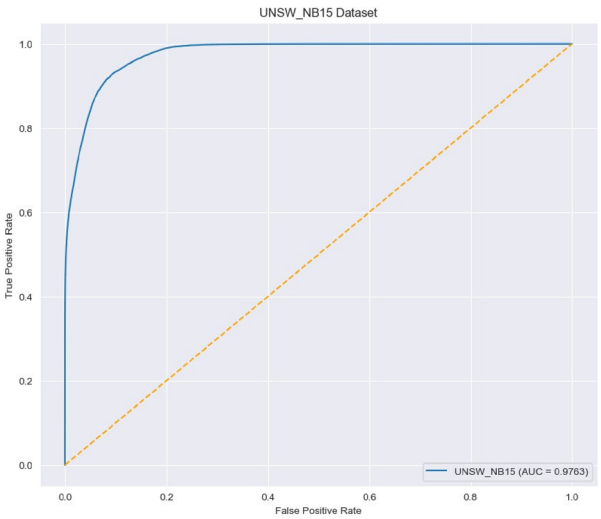
Fig. 8 Training, validation accuracy and loss for the two datasets

Table 9 Table Evaluation of performances for two datasets

Dataset	Accuracy	Precision	Recall	F1-Score	TPR	TNR	FPR	FNR	MCC
CICIoT 2023	99.82	97.83	95.91	96.86	95.91	99.94	00.06	04.09	96.77
UNSW_NB15	95.55	88.78	57.77	70.06	57.79	99.19	00.81	09.49	79.26



(a) ROC curve for CICIoT 2023 dataset



(b) ROC curve for UNSW_NB15 dataset

Fig. 9 ROC curves for two datasets

Table 10 Table Comparison of performance of four techniques for the UNSW_NB15 dataset

Reference	Dataset	Intrusion Detection Technique	Accuracy
[14]	UNSW_NB15	Deep Neural Network	93%
[23]	UNSW_NB15	Variational long short-term memory (VLSTM) learning mode	86%
Our Work	UNSW_NB15	DeepCLG	95.55%

single output layer. These optimal network structures were applied to both datasets. The ReLU activation function and SoftMax activation function were utilized in this study. Each dataset underwent 20 training epochs, with learning rates set at 0.01–0.001 and dropout rates of 0.2 and 0.3.

The model's performance can be compared between the two datasets by employing the identical set of generic multi-class labels utilized in the CICIoT 2023 dataset. The performance evaluation outcomes are displayed in Table 5 and depicted via the Receiver Operating Characteristic (ROC) AUC curve. Additionally, the accuracy of each class in predicting cyber-attacks is represented by a heatmap in the context of binary classification. In binary classification tasks, data associated with attack classes are inherently considered anomalies and abnormal, whereas in the context of multi-class classification, in contrast to binary classification, the system can identify the specific type of threat associated with the data. The confusion matrix illustrated in Fig. 6, illustrates how well the model categorizes different types of attacks in the IIoT and IoT datasets with better accuracies. The model's high accuracy is indicated by the occurence of true positive predictions in the diagonal cells. The proposed deepCLG model successfully detects various attack types in binary and multi-class classification tasks, highlighting its resilience in the IIoT cybersecurity domain. Similarly, the area under the ROC AUC curve is calculated, the AUC value indicates a model's effectiveness in threat detection. A high AUC value signifies the model's effectiveness in distinguishing between output classes. The ROC AUC curve for the proposed process is presented in Figs. 6(c), and 7(c) for the CICIoT 2023 and UNSW_NB15 datasets.

Figure 8(a) and (b) present the hyperparameters and the relationship between training accuracy and loss. The UNSW_NB15 dataset required a longer training process to bring the loss within acceptable bounds. The accuracy and loss visualizations for both datasets are displayed in Fig. 8(a) and (b) respectively. Moreover, the DeepCLG hybrid learning model demonstrates reliability and robustness by consistently achieving high performance on both datasets.

Our model's implementation was not taken lightly. We prioritized time efficiency, especially in the context of an Intrusion Detection System (IDS). The training time required for our DeepCLG model to analyze the CICIoT 2023 dataset was meticulously measured despite the inevitable disparities in hardware and software. The architecture was implemented using TensorFlow, a choice that enhances the credibility of our results, as shown in Table 6.

Timeliness is crucial in contemporary Intrusion Detection Systems (IDS); therefore, we further evaluated the duration of training needed for the UNSW_NB15 dataset. The findings of this comparative analysis are presented in Table 7. Our model's ability to consistently achieve high accuracy levels is thus encouraging while substantially reducing the required training time.

Table 8 presents the findings from our generic multi-class analysis of the UNSW_NB15 dataset, conducted using the DeepCLG hybrid learning model.

The results derived from the DeepCLG hybrid learning model, which encompasses deep learning techniques, as depicted in Table 9, indicate superior performance on the CICIoT 2023 dataset compared to the UNSW_NB15 dataset.

The ROC-AUC scores for all vectors in the datasets are nearly equal to one for CICIoT 2023 dataset and approaching 1 in the case of the UNSW_NB15 dataset (see Fig. 9). In Fig. 9, the receiver operating characteristic (ROC) curves illustrate the performance of the DeepCLG hybrid learning model for the two datasets, focusing on the True Positive Rate (TPR), and False Positive Rate (FPR). The shaded blue areas under the curves represent the accuracy levels of the model for both the CICIoT 2023 dataset and the UNSW_NB15 dataset, which are approximately 99.99% and 97.79%, respectively. Both images show that the blue curve is approximately at its peak in the top left corner. The performance is nearly perfect or near-optimal, where all positive and negative observations are accurately classified.

5.5 Comparative analysis with the relevant studies

Furthermore, we have comprehensively compared the results generated by our novel DeepCLG model and those obtained from previous deep learning-based Network Intrusion Detection Systems (NIDSs) [14, 23] for the UNSW_NB15 dataset. This comparison, detailed in Table 10, focuses on the most recent planned designs from 2018 to 2023. Many researchers have used this dataset in their research inquiries, evaluating the effectiveness of their models through the quantification of the attack detection accuracy. Our DeepCLG hybrid learning model, a unique addition to this landscape, has demonstrated higher accuracy in detecting attacks than current advanced machine learning classifiers, further establishing its superiority.

5.6 Merits and demerits of the proposed model

The proposed deepCLG hybrid learning presents several benefits. Firstly, it can distinguish between normal and malicious actions in an IIoT setting. This capability is attained through a training process incorporating normal and attack behaviours, bolstering the system's competence to identify advanced cyber-attacks. Secondly, it offers an automated feature engineering process that minimizes the time and effort needed and improves its effectiveness in real-world applications. Ultimately, the performance depends on parameters adjusted solely during the training phase, enabling it to handle noisy and outlier data [10].

A constraint of this approach is the complexity associated with parameter selection during the training phase. Nevertheless, this drawback is deemed insignificant, considering the accessibility of sophisticated and high-performance hardware [10].

6 Conclusion

This research presents a new network intrusion detection system (NIDS) that is based on deepCLG hybrid learning. The NIDS is built upon a capsule network integrated with combined deep learning methodologies, including CNN, LSTM, and GRU. The proposed model successfully detects cyberattacks with high performance measures in two distinct IIoT network datasets. The proposed model detected various attacks, including denial of service (DoS), Mirai, reconnaissance, web-based, spoofing, etc. The resulting DeepCLG hybrid learning model's performance is evaluated using the CICIoT 2023 and the UNSW_NB15 datasets. We have deployed our suggested model in TensorFlow and extensively evaluated its functionalities. The results of our study demonstrate that our methodology exhibits notable levels of accuracy, precision, recall, f1-score, detection rates, and minimal false alarms, while also resulting in decreased training durations, as illustrated in Tables 5, 6, 7, 8, and 9.

Although our model has shown excellent results, there are still a few imperfections. More testing on datasets other than CICIoT 2023 and UNSW NB_15 is required to confirm the model's generalization to actual IIoT environments. A major limitation of this study is that although the model exhibits high performance on balanced datasets, additional research is necessary to determine its scalability, real-time applicability, and resilience against changing and sophisticated cyberattacks in extensive IIoT contexts. Furthermore, the computational complexity of the hybrid architecture can impact scalability, and its ability to adjust to changing real-time cyberattacks was not examined, which calls for furhter investigation in future studies.

Acknowledgements Qawsar Gulzar acknowledges the University Grants Commission (UGC), India, for the Junior Research Fellowship [200510090572/(UGC NET June 2020)].

Author Contributions Conceptualization: QG; Methodology: QG; Data Curation: QG; Formal analysis: QG and KM; Investigation: QG; Validation: QG and KM; Visualization: QG; Writing-Original Draft: QG; Writing-Review & Editing: QG and KM.

Funding Not applicable.

Data Availability The manuscript provides all the data in figures and tables.

Declarations

Conflict of interest No Conflict of interest exists as reported by the authors.

References

1. Jeong S, Na W, Kim J, Cho S (2018) Internet of things for smart manufacturing system: Trust issues in resource allocation. IEEE Internet Things J 5(6):4418–4427
2. Newman P (2020) The internet of things 2020: Hereâ's what over 400 iot decision-makers say about the future of enterprise connectivity and how iot companies can use it to grow revenue. Business Insider, pp. 1–6
3. Anwaar F, Iltaf N, Afzal H, Nawaz R (2018) Hrs-ce: A hybrid framework to integrate content embeddings in recommender systems for cold start items. J Comput Sci 29:9–18
4. Lee J, Bagheri B, Kao H-A (2015) A cyber-physical systems architecture for industry 4.0-based manufacturing systems. Manuf Lett 3:18–23
5. Sengupta J, Ruj S, Bit SD (2020) A comprehensive survey on attacks, security issues and blockchain solutions for iot and iiot. J Netw Comput Appl 149:102481
6. Shahin M, Chen FF, Hosseinzadeh A, Bouzary H, Rashidifar R (2022) A deep hybrid learning model for detection of cyber attacks in industrial iot devices. Int J Adv Manuf Technol 123(5):1973–1983
7. Ferrag MA, Shu L, Djallel H, Choo K-KR (2021) Deep learning-based intrusion detection for distributed denial of service attack in agriculture 4.0. Electronics 10(11):1257
8. Zhang Q, Yang LT, Chen Z, Li P (2018) A tensor-train deep computation model for industry informatics big data feature learning. IEEE Trans Indust Inf 14(7):3197–3204
9. Diro AA, Chilamkurti N (2018) Distributed attack detection scheme using deep learning approach for internet of things. Future Generat Comput Syst 82:761–768
10. Muna A-H, Moustafa N, Sitnikova E (2018) Identification of malicious activities in industrial internet of things based on deep learning models. J Inf Security Appl 41:1–11
11. Vinayakumar R, Alazab M, Srinivasan S, Pham Q-V, Padannayil SK, Simran K (2020) A visualized botnet detection system based deep learning for the internet of things networks of smart cities. IEEE Trans Indust Appl 56(4):4436–4456
12. Parra GDLT, Rad P, Choo K-KR, Beebe N (2020) Detecting internet of things attacks using distributed deep learning. J Netw Comput Appl 163:102662
13. Manimurugan S, Al-Mutairi S, Aborokbah MM, Chilamkurti N, Ganesan S, Patan R (2020) Effective attack detection in internet

of medical things smart environment using a deep belief neural network. IEEE Access 8:77396–77404
14. Koroniotis N, Moustafa N, Sitnikova E (2020) A new network forensic framework based on deep learning for internet of things networks: A particle deep framework. Future Generation Comput Syst 110:91–106
15. Otoum Y, Liu D, Nayak A (2022) Dl-ids: a deep learning-based intrusion detection framework for securing iot. Trans Emerging Telecommun Technol 33(3):e3803
16. Altunay HC, Albayrak Z (2023) A hybrid cnn+ lstm-based intrusion detection system for industrial iot networks. Eng Sci Technol Int J 38:101322
17. Khanday SA, Fatima H, Rakesh N (2023) Implementation of intrusion detection model for ddos attacks in lightweight iot networks. Expert Syst Appl 215:119330
18. Khan IA, Pi D, Abbas MZ, Zia U, Hussain Y, Soliman H (2022) Federated-srus: A federated-simple-recurrent-units-based ids for accurate detection of cyber attacks against iot-augmented industrial control systems. IEEE Internet Things J 10(10):8467–8476
19. Javeed D, Saeed MS, Adil M, Kumar P, Jolfaei A (2024) A federated learning-based zero trust intrusion detection system for internet of things. Ad Hoc Networks 162:103540
20. Sabour S, Frosst N, Hinton GE (2017) "Dynamic routing between capsules. Adv Neural Inf Process Syst vol. 30
21. Chen W, Fang F, Wang P, Kan J, Li W, Wu W (2022) "Research on medical text classification based on biobert-gru-attention. In: 2022 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA), pp. 213–219, IEEE
22. Alguliyev R, Imamverdiyev Y, Sukhostat L (2021) Hybrid deep-gcl model for cyber-attacks detection on cyber-physical systems. Neural Comput Appl 33(16):10211–10226
23. Zhou X, Hu Y, Liang W, Ma J, Jin Q (2020) Variational lstm enhanced anomaly detection for industrial big data. IEEE Trans Indust Inf 17(5):3469–3477
24. Neto ECP, Dadkhah S, Ferreira R, Zohourian A, Lu R, Ghorbani AA (2023) Ciciot 2023: A real-time dataset and benchmark for large-scale attacks in iot environment. Sensors 23(13):5941

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.